Projet d'Algorithmie et de Programmation

YAHIA-AMAR Clément, PREVEL Noé et HURSTEL Mathis

Décembre 2024

Contents

1 Introduction **2**

2 Manuel d'utilisateur **2**
2.1 Application première 2
2.2 Compilation 2
2.3 Procédures de test 3
2.4 Commandes secondaires 4

3 Choix de conception **4**

4 Fonctionnement du trinôme **6**
4.1 Organisation temporelle et logistique 6
4.2 Méthodologies de travail 6
4.3 Répartition des rôles 7

5 Résultats et analyses **7**

6 Bilan du projet **10**
6.1 Difficultés rencontrées 10
6.2 Perspectives d'amélioration 10
6.3 Conclusion 10

1 Introduction

L'objectif de ce projet est de concevoir et d'implémenter un vérificateur orthographique en langage C.

Celui-ci devra parcourir un texte et afficher les mots n'appartenant pas à un dictionnaire construit au préalable, à partir d'un fichier donné.

Ce projet a pour but de mettre en œuvre différentes structures de données afin d'optimiser les performances du vérificateur orthographique, notamment pour des textes très volumineux. Nous serons donc amenés à réaliser des études temporelles et de gestion de mémoire pour chaque structure de données, dans le but de comparer leur efficacité lorsque ces dernières sont soumises à différents besoins de la part de l'utilisateur.
Les structures de données à implémenter incluent des tableaux redimensionnables, des ensembles, des arbres lexicographiques et des tries. Elles seront utilisées pour le stockage du dictionnaire et la recherche optimisée des mots à l'intérieur.

Le programme prendra en entrée deux fichiers :

- le fichier du dictionnaire (par exemple `dico1.txt`)
- un fichier texte à vérifier (par exemple `a_la_recherche_du_temps_perdu.txt`).

À partir de ces fichiers, notre programme sera en mesure d'identifier et d'afficher les mots non présents dans le dictionnaire.

2 Manuel d'utilisateur

Afin qu'un utilisateur tiers puisse s'y retrouver au sein du projet, ce paragraphe répertorie les différentes commandes nécessaires à la prise en main du programme.
Veuillez trouver dans les sous-parties suivantes le détail du Makefile fourni avec le rendu GIT.

2.1 Application première

Afin de répondre purement aux besoins de ce projet, c'est à dire comparer les mots d'un fichier avec la base de données d'un dictionnaire, afin de lister ceux qui n'y figurent pas, on peut exécuter dans la racine du projet la commande :

```
< make testTempsPerdu >
```

2.2 Compilation

Ces commandes sont à écrire dans un terminal, depuis la racine du projet. Elles permettent de "traduire" le code d'un langage de programmation (en C ici) à un fichier binaire, compréhensible pour le processeur, afin de vérifier que le code ne comporte aucune erreur.

- `make` : Compile tout le projet ainsi que les tests. C'est la commande par défaut qui permet de prendre l'intégralité des fichiers.
  Au vu des nombreuses inclusions, rendant chaque fichier très dépendant des autres, c'est la commande que nous recommandons d'utiliser si la compilation est réalisé sans objectif de débogage très précis.

- `make programs/%` : Compile un programme spécifique dans le répertoire `programs/` à partir du fichier source correspondant.
- `make %` : Compile un fichier source `.c` en fichier objet `.o`.
- `make tests` : Compile tous les tests définis dans le répertoire `test/`.
- `make test/%` : Compile un test spécifique dans le répertoire `test/`.

2.3 Procédures de test

A. Structure des tests :

Pour les tests, nous avons choisi d'utiliser une forme classique avec des `macros` afin d'être plus efficace et plus rigoureux.
Voici la forme en question.

```
#define + :

- ENTERING() fprintf(stderr, "\n## Entering in %s function\n", __func__);
- VALID() fprintf(stderr, "<= Test : We try valid parameters, must return no errors\n");
- INVALID() fprintf(stderr, "<= Test : We try invalid parameters, must return errors\n");
- SUCCESS() fprintf(stderr, "=> Test passed with success\n");
- EXIT() fprintf(stderr, "Tests of file %s passed with success\n", __FILE__);
         return EXIT_SUCCESS;


      int test_proto() {
          ENTERING();
          INVALID();
          SUCCESS();
          VALID();
          SUCCESS();
          return 1;
      }
```

Cette forme explicite permet de toujours prendre en compte les cas avec des erreurs dans les arguments.
Dans les tests, nous vérifions que les erreurs renvoyées soient les bonnes avec des `assert`.
On compare ensuite les résultats obtenus avec ceux attendus, également avec des `assert`.

B. Exécution des tests :

Nous détaillons dans cette sous-partie les commandes pour compiler ET exécuter les pincipaux tests du projet, afin de vérifier que le programme exécute bien ce qui lui est demandé, et ce proprement.
Exécuter la commande désirée dans le terminal :

- `make tests@check` : Compile et exécute tous les tests, tout en vérifiant les fuites de mémoire via AddressSanitizer (ASan) et UndefinedBehaviorSanitizer (UBSan).

- `make tests@debug` : Compile et exécute tous les tests dans un débogueur (`gdb`).
- `tests@check-recursive` : Exécute tous les tests de manière récursive et vérifie les fuites de mémoire. Arrête si un test échoue.
- `make tests@debug-recursive` : Compile et exécute tous les tests de manière récursive dans un débogueur.
- `make tests_oracle@check` : Exécute le script `tests_oracle/check_oracles.sh` pour vérifier que tous les tests "oracle" se sont bien exécutés.

2.4 Commandes secondaires

- `help` : Affiche l'aide sur les commandes et les options disponibles.
- `make clean` : Supprime tous les fichiers objets (`.o`) et les programmes compilés dans `programs/`. Nettoie également les tests compilés.
- `make deep-clean` : Effectue un nettoyage complet, en supprimant les fichiers objets, les fichiers générés par `make`, ainsi que d'autres fichiers temporaires comme les fichiers `*~` et `._*`.

3 Choix de conception

Le choix des structures de données est crucial pour ce projet, car il aura un impact direct sur l'efficacité du programme. Nous avons opté pour plusieurs approches, que nous avons implémentées et comparées. Voici les principales structures choisies :

- **Tableau redimensionnable avec recherche séquentielle** : cette approche consiste à utiliser un tableau qui peut croître dynamiquement. La recherche des mots dans le tableau se fait en O(n) car la seul manière est de vérifier dans tout le tableau. Lorsqu'on ajoute un élément dans un tableau redimensionnable, il y a deux cas possibles :
  - **Ajout sans redimensionnement** : O(1).
  - **Ajout avec redimensionnement** : Si le tableau est plein, on doit le redimensionner, ce qui prend O(n), où n est la taille actuelle du tableau.

  Soit m le nombre total d'ajouts. Lors des redimensionnements, la taille du tableau double à chaque fois, et le coût total des redimensionnements est :

  $$1 + 2 + 4 + \cdots + m/2 = O(m)$$

  Les ajouts sans redimensionnement sont O(1) chacun. Ainsi, le coût total des ajouts est O(m), et le coût moyen par ajout est :

  $$\frac{O(m)}{m} = O(1)$$

  Donc, l'ajout moyen dans un tableau redimensionnable est en O(1).

- **Tableau redimensionnable avec recherche dichotomique** : ici, après avoir trié le tableau, la recherche se fait à l'aide de la méthode de recherche dichotomique (ou binaire), qui est plus rapide qu'une recherche séquentielle pour un grand nombre d'éléments.
  Complexité : La recherche se fait en O(log(n)) et l'ahout d'un élément en O(n).

- **Ensembles** : nous avons utilisé une structure d'ensemble pour stocker les mots du dictionnaire, permettant une recherche plus rapide. L'implémentation d'un ensemble repose sur des structures telles que des tables de hachage.

- **Table de hashage** : cette structure est créée par un tableau de listes de pointeurs afin de gérer les collision et de pouvoir se servir d'objets les plus généraux possible. La complexité maximale pour l'ajout et la recherche est O(n), mais si la taille du tableau est choisie judicieusement, alors on passe à une complexité en O(1).
  En effet, la complexité pour la recherche est O(n/taille de la table de hachage). Due à la façon dont sont implémentés les tables de hachages, on a n environ égale à la taille de la table de hachage.

- **Arbre Patricia** : cette structure est, comme son nom l'indique, un arbre qui va servir au stockage d'un ensemble de string en ajoutant les lettre une par une et donc en regroupant le début des mot qui ont un préfixe commun. Cela est très utile, surtout dans une langue comme le français où les préfixes communs sont bien plus courants que dans un tirage aléatoire de lettres.
  Complexité : Pour n mots de taille k,
  L'ajout est en O(k) (O(n*k) pour n mots)
  La recherche est en O(k).

- **Arbre compressé** : cette structure se base sur l'arbre Partricia , auquel on rajoute en plus une autre optimisation possible. En effet si deux préfixes ont tous leurs suffixes en commun, alors on les fusionne pour gagner de l'espace mémoire supplémentaire. En revanche, avoir de multiples pointeurs vers un même objet rend la libération de la mémoire plus compliquée pour éviter les doubles free(). Pour se faire, on se sert d'un compteur par noeud qui indique combien de pointeurs pointent vers ce dernier. On a alors recourt au free() seulement si ce compteur a atteint 0, soit qu'on a "coupé tous les fils auxquels il était attaché".
  Complexité : La complexité asymptotique reste la même que celle de l'arbre Patricia normal malgrès un surcout pour la recherche de suffix

- **trie tas** : La généricité est implémentée ici par des macro en faisant une conversion explicite entre void* et le type réel via un cast et par le passage de size pour tout les calcules. Cela nous permet de manipuler des données sans connaître leur type. Tout les calcul des décalages mémoire ont été fait par arithmétique de pointeurs. - Mécanisme de comparaison générique via un pointeur de fonction CompareFunc permet de passer une fonction de comparaison adaptée au type. Cette fonction transforme les void* en type spécifique par cast et permet une comparaison dynamique sans connaissance préalable du type. Le point technique crucial est la gestion mémoire : chaque opération (comparaison, échange) utilise size pour calculer précisément les adresses mémoire, permettant de traiter n'importe quel type avec la même logique algorithmique.

La complexité de ce trie est O(nlog(n)) la meilleur possible pour un trie par comparaison.

Chacune de ces structures offre des avantages et des inconvénients en fonction de la taille du dictionnaire et du texte à vérifier. Le but de ce projet est d'analyser les performances de ces différentes structures et de déterminer laquelle est la plus appropriée pour chaque scénario.
En effet, la langue d'écriture ou encore le style littéraire sont des facteurs qui vont fortement influencer la nature des mots présents dans le fichier, et donc les vérifications nécessaires.

4 Fonctionnement du trinôme

4.1 Organisation temporelle et logistique

En raison du projet GL qui commençait début Décembre, nous avons tous les trois souhaité se pencher assez vite sur le projet afin d'en effectuer le travail principal pour mi Novembre.

Nous nous sommes fixés plusieurs deadlines claires, tant dans l'idée de chacun avoir des objectifs que de s'adapter aux façons de travailler de chacun et de ne ralentir personne. Nous avons donc mis en place un GIT, divisé et organisé clairement avec les fichiers classiques (src, include, test...) et communiquions beaucoup sur nos avancées personnelles afin de ne pas se marcher dessus.

4.2 Méthodologies de travail

A. Travail préalable :

Des projets aussi conséquents, que ce soit par leur volume de code, leur nombre de fichier ou leur imbrication mutuelle, demande une rigueur particulièrement exigeante.
En effet, comme nous le détaillerons plus loin, un mauvais choix conceptuel en début de projet impose de réécrire un nombre non négligeable de fichiers avec leurs tests correspondants.

Pour éviter ce genre d'erreurs d'organisation, nous avons au préalable posé le projet à l'écris afin d'en mesurer l'ampleur et les points d'attention.
Cette réflexion, notamment sur l'ordre de priorité de programmation des fonctions, la répartition du travail au sein du trinôme et le choix des structures, nous a permis de gagner un temps non négligeable.

B. Utilisation du GIT :

Ayant mis un point d'honneur à travailler sur un projet ne comportant aucune fuite mémoire et validant tous les tests mis en place, nous avons eu recours à l'utilisation de `branches`.
Ces dernières permettent d'évoluer à notre rythme sur nos fonctions respectives et de push

nos programmes, même si ils ne compiles pas parfaitement, sans corrompre les fichiers des autres membres du trinôme quand ils pull.
Cette politique de ne merge sur le main que des fichiers parfaitement codés ralentit parfois l'avancement mais évite de perdre de nombreuses heures à déboger quelques erreurs sur des centaines de lignes de code.

C. Travail personnel/collectif :

Nous avons travaillé de plusieurs manières tout au long de ce projet :

- Les séances de TD où nous avions l'occasion d'avancer sur l'implémentation progressive des structures nécessaires à la bonne réalisation du projet.
  Nous nous étions fixé de tout le temps venir, au moins 2 heures, afin de faire un bilan des avancées et de se chacun se mettre à jour sur le travail à effectuer. C'était également l'occasion d'échanger à trois sur nos difficultés et d'échanger avec les professeurs encadrants sur les points de flou.

- Chez soi, selon le rythme de chacun. Les vacances de la Toussaint nous ont d'ailleurs aidé à bien mettre le projet en route.

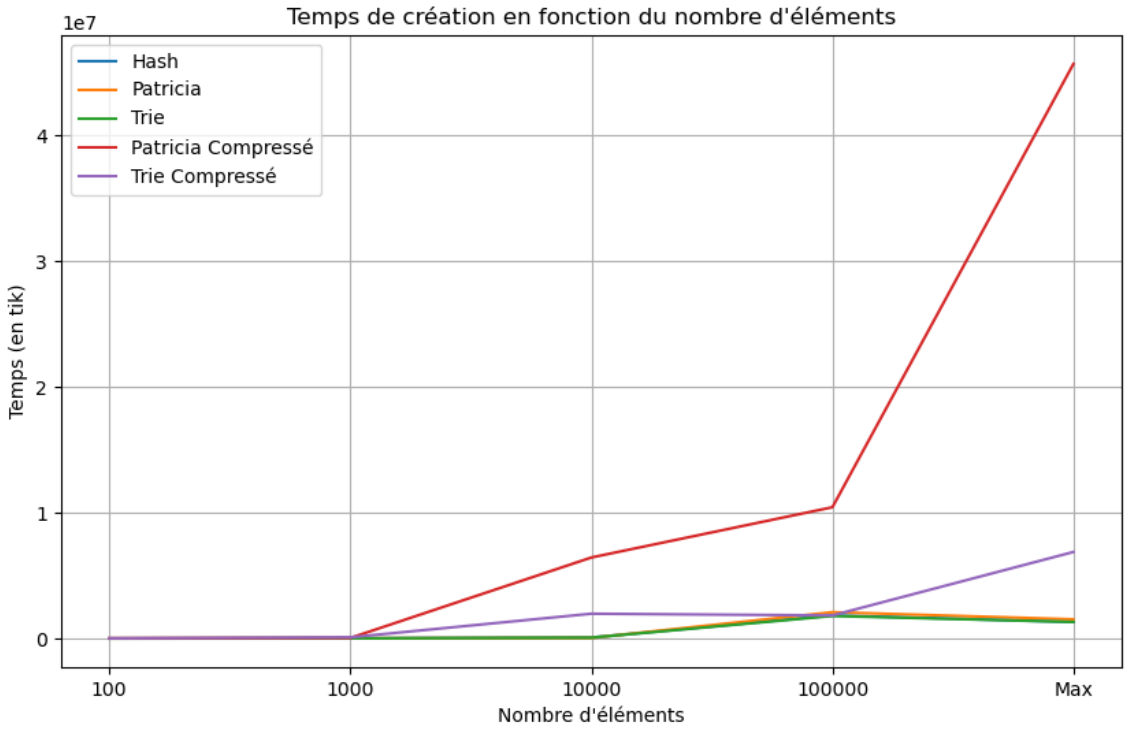
- En salle informatique, en particulier vers la fin du projet, où nous passions tout notre temps libre afin de travailler sur les machines de l'école et d'être avec les autres élèves de la SEOC.

4.3 Répartition des rôles

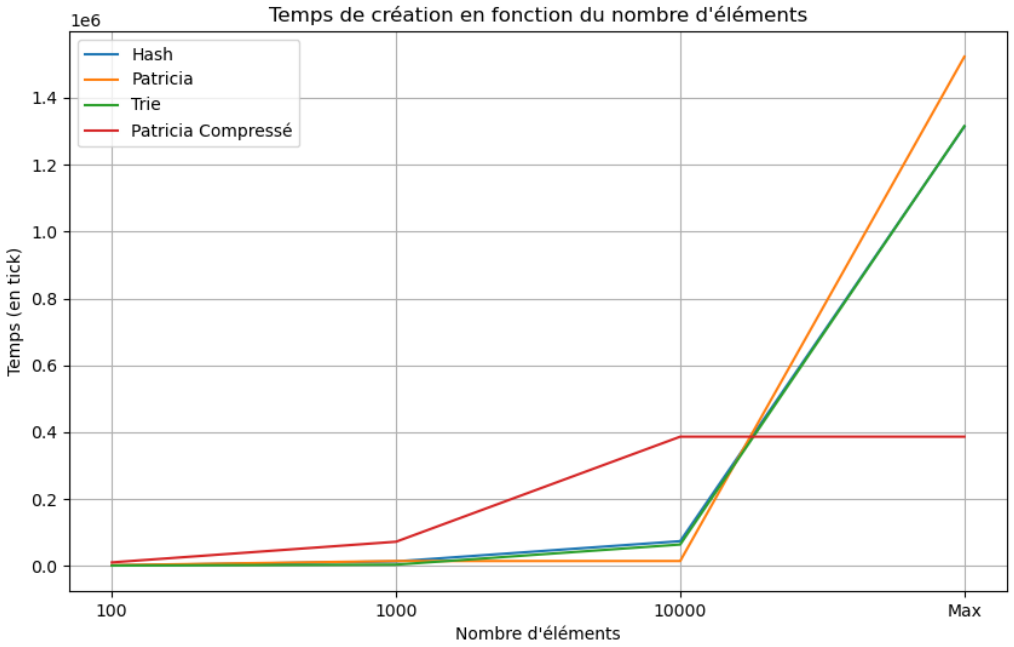
Au sein du groupe, en plus des structures du TD, le travail a été plutôt bien réparti :
Bien que Clément ait plus contribué aux structures et à l'organisation des fichiers dans le projet ainsi qu'à la forme des tests, Noé et Mathis se sont concentrés sur l'implémentation des structures et des tris et ont rédigé le rapport.

5 Résultats et analyses

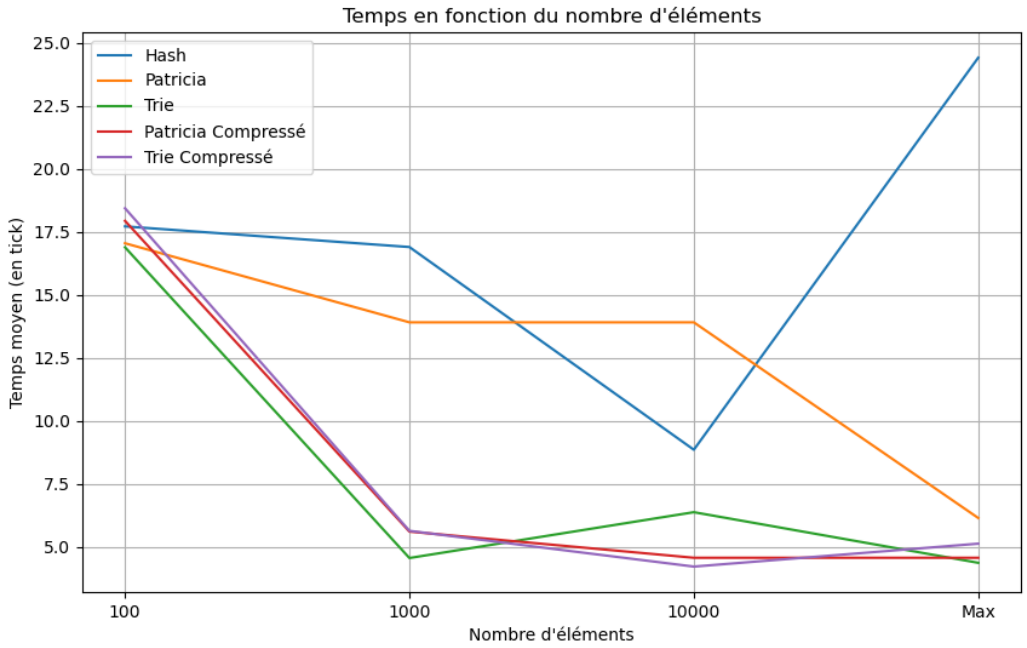
Pour l'analyse des différents moyen de stockage de l'information, nous travaillerons avec une échelle semi-log afin de couvrir plusieurs ordres de grandeur et de voir l'efficacité de l'algorithme sur un large intervalle.



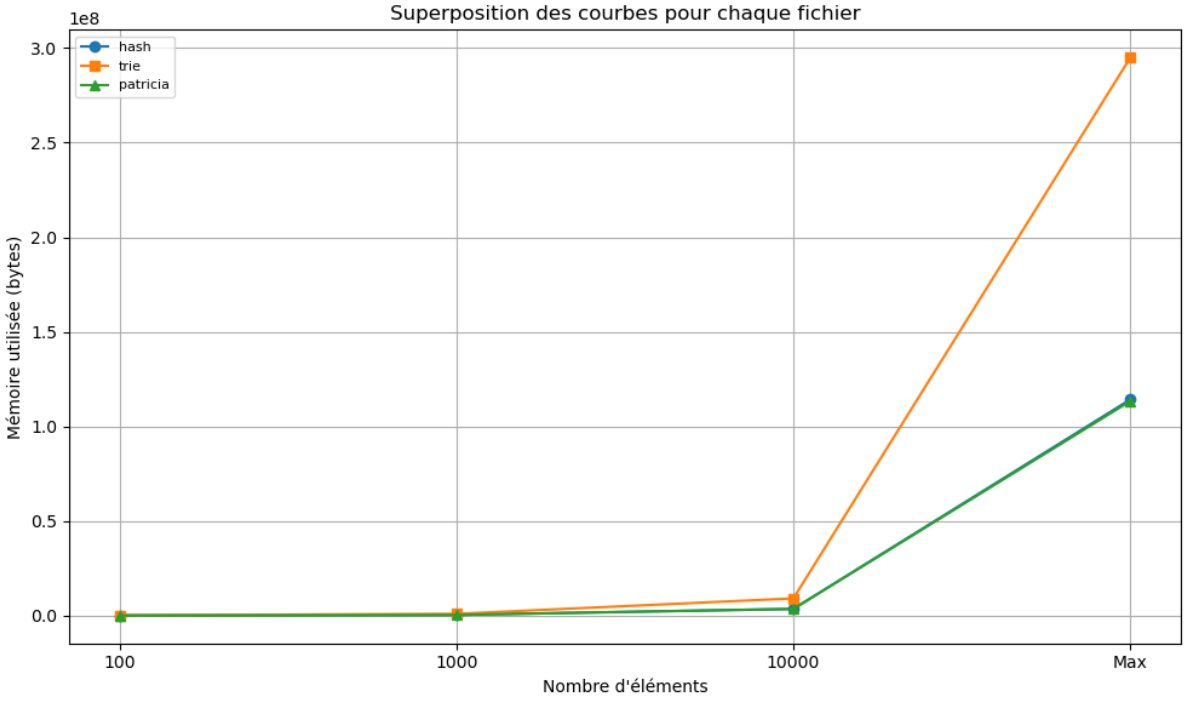
Sur ce premier schéma on peut remarquer qu'il y a une forte augmentation du temps (max = 300 000) mais pas proportionnel



Ici on retrouve le même schéma, bien qu'il ne faille pas considérer Patricia compressé, qu'on a changé pour zoomer et pouvoir comparer les autres entre eux. Leur complexité est sensée être la même (entre Patricia et Trie) étant donné que seul l'implémentation diffère mais l'algorithme reste le même.
On retrouve effectivement que le coût temporel entre ces deux implémentation est très proche.



Dans ce troisième graphique, on regarde le temps d'ajout par élément. On obtient un résultat qui semble aléatoire. Cela peut être dû au fait qu'on faisait la moyenne du temps d'ajout sur 100 éléments. Un nombre plus grand aurait pu être souhaitable. On remarque tout de même une tendance à l'augmentation du temps pour le Hash avec 300 000 éléments. Cela peut s'expliquer par ses listes d'adjacence qui commencent à être remplies. Un tableau initialement plus grand aurait été préférable, mais si on considère qu'on ne connaît pas l'ordre de grandeur de l'information qu'on va stocker, ce tableau plus grand aurait rendu la création plus lente, donc le code plus lent pour des valeurs plus petites de n.



Dans ce dernier graphique on a la différence pour la complexité spatial. On remarque que même si Trie et Patricia sont de même nature, la différence d'implémentation change

beaucoup le résultat d'un point de vue spacial. La création d'un tableau entier pour chaque noeud avec une grosse partie non utilisé rend le Trie moins bon d'un point de vue mémoriel.

6 Bilan du projet

6.1 Difficultés rencontrées

L'un ds principaux obstacles a été le choix de la structure de liste.
Dans un premier temps, nous avons utilisé une liste simplement chaînée non circulaire, que nous avons intégrée à notre code.
Cependant, nous avons souhaité par la suite la transformer en liste chaînée circulaire afin de faciliter l'implémentation d'une file. Cette première version structurelle ayant déjà été utilisée dans d'autres parties du programme, elle a nécessité de reconstruire un certain nombre de fonctions selon ce nouveau modèle.

La génération de résultats concluants et analysables a également requis un travail conséquent, tant au niveau des mesures/tests à effectuer sur les différentes structures qu'au niveau des échantillonnages choisis pour obtenir des courbes pertinentes.
Il a fallu trouver les simulations adéquates à la réponse temporelle et mémoire de chaque structure afin de pouvoir travailler proprement sur les graphiques. Le besoin de comparer les courbes sur des échelles identiques nous à imposer de reprendre à de maintes reprise les calculs de complexité.

6.2 Perspectives d'amélioration

Nous avons donc décidé de conserver les deux versions du projet (liste non-circulaire et liste circulaire). Pour éviter ce type de problème dans un contexte de projet, une solution serait de formaliser et de typer toutes les structures dès le début, en fonction de leur rôle, et de trouver un juste milieu pour être efficaces tout en permettant des modifications futures.

Une autre amélioration envisageable serait de rédiger le rapport tout au long du projet, afin d'y répertorier l'avancement, l'organisation et les obstacles.
Bien que notre gestion du temps fût tout-à-fait satisfaisante, puisque nous avons commencé sérieusement le projet assez tôt et que nous l'avons fini largement dans les temps, nous avons laissé le rapport sur le côté. Il était donc à faire pendant la période des examens/rendus de projet, en supplément au fait que les détails de conception sont autant précis et véridiques qu'ils sont frais et récents.

6.3 Conclusion

Au final, le projet fonctionne comme prévu.
Les structures de données ont été implémentées avec succès, et après des tests approfondis, aucun problème de mémoire n'a été détecté avec Valgrind. Tous les tests ont été réalisés et sont concluants.

Nous avons également utilisé deux types de généricité : par pointeur pour la plupart des programmes, et par macro pour le cas du trie par tas.

Cela nous a permis de comprendre comment marchent les différents type de généricité.

Enfin, les choix techniques se sont révélés efficaces pour répondre aux exigences. le projet est fonctionnel et robuste dans son état actuel et nous sommes tout-à-fait satisfaits du travail rendu malgré quelques observations critiques citées ci-dessus.